# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

| | | | |
|---|---|---|---|
| **Course Code:** | CSA10 / CSA1063 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO2 | **Assessment:** | Assessment Tool 2 – Architecture Design Assignment |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO2 Statement:** | *Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills* | | |
| **Student Name:** | Preksha Prakash | **Reg. No.:** | 1922260048 |
| **Date:** | 25-10-2026 | **Duration:** | 60 minutes |

**Code of Conduct**

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

## Annexure A – Architecture Design Assignment

**Instructions to Students:**

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

## 5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

## 6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

# 1. System Requirement Analysis

## 1.1 System Objectives and Domain Context

In modern enterprise software engineering, business systems operate under strict service level agreements that demand uninterrupted continuous availability. High-concurrency web applications, distributed microservices, and customer-facing digital platforms rely heavily on underlying relational and non-relational database management systems to store operational data. As software applications continuously evolve through agile development lifecycles, the database structures supporting them must undergo frequent schema alterations, such as adding structural attributes, splitting large tables, normalizing relations, or altering data types.

Executing structural modifications directly on live production databases introduces immense operational risks. Traditional data definition language operations often acquire exclusive table locks, which reject incoming application read and write transactions, resulting in severe latency spikes, service degradation, or catastrophic downtime. Furthermore, when updated database schemas are deployed before all running application instances across a distributed server farm are updated, legacy application code frequently crashes when attempting to access non-existent or renamed database fields.

The primary objective of the Zero-Downtime Database Migration and Schema Evolution Platform is to establish an automated, enterprise-grade architectural framework that governs, validates, and orchestrates schema changes across distributed production environments. By enforcing dual-writing patterns, multi-stage deployment workflows, real-time telemetry tracking, and pre-deployment static analysis, the platform ensures that database schemas evolve smoothly alongside continuous application deployments without causing user-facing downtime, service incompatibilities, or data corruption.

## 1.2 Functional and Non-Functional Requirements

The functional capabilities of the proposed system encompass the full operational lifecycle of database schema changes. The platform must incorporate an automated static analysis engine capable of parsing SQL scripts and migration code to detect high-risk or destructive database commands, such as immediate table drops, non-default column additions, or unindexed foreign key creations. It must execute backward compatibility simulations that evaluate how both legacy and incoming application code versions interact with modified database structures.

To safely execute structural changes without acquiring prolonged table locks, the platform must orchestrate multi-stage expand-and-contract execution workflows. In addition, the system must interface seamlessly

with external continuous integration and continuous deployment pipelines, track real-time database telemetry during execution, and automatically execute safe rollback sequences whenever database performance limits are breached.

From a non-functional perspective, the system must fulfill rigorous performance, security, and operational quality metrics. Performance requirements dictate that pre-deployment static analysis and compatibility checks for batch migration files containing up to fifty schema statements must complete within fifteen seconds to prevent build pipeline bottlenecks. Security controls mandate that all sensitive database credentials, connection strings, and administrative tokens must be encrypted using standard AES-256 encryption at rest and protected via TLS 1.3 during network transmission.

Reliability requirements demand that the platform utilize distributed state management and transactional locking, ensuring that unexpected network partitions or node failures during deployment do not leave production databases in partial or corrupted states. The platform orchestrator must maintain ninety-nine point nine nine percent system availability and scale horizontally to manage concurrent migration pipelines across up to one hundred distinct enterprise database clusters.

# 2. Selection of Software Architecture

## 2.1 Architectural Style Selection

To fulfill the operational demands of distributed database orchestration, high concurrency, fault isolation, and asynchronous processing, a **Hybrid Microservices and Event-Driven Architecture** has been selected as the foundational software architecture. This architectural paradigm combines modular, loosely coupled microservices dedicated to specific operational tasks (such as static analysis linting, execution orchestration, and telemetry tracking) with a high-throughput event-driven messaging core (such as Apache Kafka or RabbitMQ) that governs inter-service communication and pipeline events.

## 2.2 Technical Justification

Selecting a hybrid microservices and event-driven architecture provides significant technical advantages over traditional monolithic or simple layered architectures. Database migration workflows are inherently asynchronous and long-running; tasks such as parsing complex DDL scripts, simulating execution against schema snapshots, backfilling data across shadow tables, and monitoring health indicators occur over variable time intervals.

An event-driven messaging backbone allows platform components to publish and consume pipeline

lifecycle events without blocking core web API threads or coupling execution components together. If a spike in developer activity occurs, resource-intensive services like the static analysis linter and compatibility simulation engine can scale out independently without impacting the core orchestration engine managing live database transitions.

Furthermore, this architectural decoupling provides strict fault isolation. If the telemetry monitoring service or web reporting dashboard experiences a temporary outage, ongoing production migration execution scripts continue safely on the target database clusters without being interrupted or corrupted.

# 3. Software Architecture Diagram and Component Design

## 3.1 Architectural Component Breakdown

The structural design of the platform is organized into four distinct horizontal tiers, ensuring a clear separation of concerns across the entire system lifecycle:

1. **Presentation & Gateway Tier:** Acts as the external entry point for human administrators and automated software pipelines. It comprises a Web Management Console for user interaction and visual analytics, alongside an API Gateway equipped with Webhook Listeners that intercept incoming deployment triggers from continuous integration platforms like GitHub Actions, GitLab CI, and Jenkins.

2. **Core Services Tier (Microservices):** Contains specialized, autonomous microservices that execute platform business logic. Key microservices include the Migration Parsing & Static Linter Service, which parses SQL statements into abstract syntax trees; the Compatibility & Simulation Engine, which tests schema backward compatibility against active application versions; the Orchestration & Execution Engine, which manages multi-stage expand-and-contract deployment sequences; the Health Monitoring & Telemetry Service, which tracks real-time database indicators; and the Rollback Manager, which handles emergency recovery.

3. **Event & Messaging Tier:** Features a distributed message broker (such as Apache Kafka or RabbitMQ) that acts as the asynchronous communication backbone. It decouples the core microservices by streaming pipeline events, risk assessment reports, and telemetry alerts across the platform.

4. **Data & Storage Tier:** Houses the platform's internal Metadata Database (PostgreSQL) for storing audit logs, migration histories, user permissions, and safety policies, while maintaining encrypted

database connectors that interface directly with external Target Production Database Clusters.

### 3.2 Textual Architecture Mapping Diagram

PRESENTATION TIER: Web Management Console | API Gateway | Webhook Listener
↓
CORE SERVICES TIER: Static Linter | Compatibility Engine | Orchestration Engine | Health Telemetry | Rollback Manager
↓
EVENT & MESSAGING TIER: Distributed Message Broker (Apache Kafka / RabbitMQ)
↓
DATA & STORAGE TIER: Platform Metadata DB (PostgreSQL) <---> Target Database Clusters

## 4. Component Interactions and Workflow

### 4.1 Component Responsibilities and Communication Protocols

Component communication across the platform uses a hybrid protocol approach to optimize speed, security, and reliability. Synchronous administrative operations—such as fetching migration audit logs, rendering dashboards on the Web Management Console, or checking pipeline execution statuses—are handled via secure RESTful HTTP/2 APIs protected by OAuth2 authentication and Role-Based Access Control (RBAC).

For core migration orchestration, components interact asynchronously via event channels managed by the distributed message broker. The API Gateway authenticates incoming pipeline webhooks and publishes a MigrationSubmittedEvent. The Migration Parsing Service consumes this event, performs static analysis, and publishes a ScriptValidatedEvent containing calculated risk scores.

The Orchestration Engine consumes approval events and communicates directly with target database clusters using encrypted native drivers (such as pgBouncer connection pools or JDBC over TLS 1.3), while the Health Monitoring Service continuously streams performance metrics over gRPC connections back to the event bus.

### 4.2 End-to-End System Deployment Workflow

The complete system operational workflow proceeds through five coordinated stages:

1. **Pipeline Trigger and Ingestion:** A software developer commits new code alongside a database migration script to a version-controlled repository. The external CI/CD pipeline sends an automated HTTP POST webhook request containing the raw DDL script to the platform's API Gateway.

2. **Parsing and Static Compatibility Analysis:** The API Gateway forwards the payload to the Migration Parsing Service, which parses the SQL statements into an abstract syntax tree. The Compatibility & Simulation Engine evaluates the code against active application queries, checking for breaking operations like immediate table drops or unindexed column additions, and publishes a structured risk assessment score.

3. **Approval and Authorization:** If critical safety policy violations are detected, the build pipeline is halted automatically with detailed remediation guidance. If the script passes or requires manual review, a Database Administrator inspects the generated diff report on the Web Management Console and signs off on execution.

4. **Phased Expand-and-Contract Orchestration:** Upon authorization, the Orchestration Engine initiates the expand phase on the target database cluster, creating additive structures like shadow columns or trigger-based dual-writing tables without locking existing data. As rolling application updates deploy new codebase instances across server farms, the platform verifies that no active connections are accessing legacy schema structures. Once full application transition is verified, the orchestrator executes the contract phase, safely pruning deprecated fields.

5. **Continuous Telemetry and Automated Recovery:** Throughout the execution window, the Health Monitoring Service polls target database performance metrics, tracking lock wait times, active query latencies, CPU utilization, and replication lag. If telemetry indicators breach safe operating thresholds or application error rates spike, the Rollback Manager immediately intercepts the pipeline, halts the execution sequence, and applies safe, non-destructive compensation transactions that restore database stability without losing newly written production records.

# 5. Quality Attribute Evaluation

Evaluating how the proposed hybrid microservices and event-driven architecture addresses core software quality attributes demonstrates its suitability for mission-critical enterprise environments:

- **Scalability:** The decoupled nature of microservices allows resource-intensive platform components to scale independently based on demand. During major software release windows, when hundreds of build pipelines submit migration scripts simultaneously, the static analysis parser and compatibility

simulation engines can scale out horizontally in containerized Kubernetes clusters without consuming resources needed by the execution engine or increasing core API latencies.

- **Security:** Security is enforced through multi-layered defense mechanisms. The API Gateway enforces strict zero-trust authentication, validating JSON Web Tokens (JWT) and enforcing Role-Based Access Control so that non-administrative accounts cannot trigger production of DDL execution. All sensitive target database connection strings, credentials, and access keys are stored in encrypted hardware security modules (HSM) using AES-256 encryption at rest, while all inter-service and database traffic is encrypted using TLS 1.3.

- **Performance:** Utilizing an event-driven messaging core ensures that long-running schema parsing, simulation testing, and data backfilling processes are handled asynchronously. This prevents API blocking and guarantees that static linting feedback is returned to external CI/CD pipelines in under fifteen seconds, keeping developer feedback loops fast and efficient.

- **Reliability:** Reliability and transactional integrity are maintained through idempotent service design and distributed transaction coordination. If a network partition, database connection drop, or container failure occurs mid-migration, the platform uses distributed locks (via Redis/ZooKeeper) and state machine tracking to safely pause execution without leaving the database in an inconsistent state or corrupting live user records.

- **Maintainability:** Modularity inherent to microservices simplifies system maintenance and continuous evolution. Platform engineering teams can update static linting rules, add support for new database engines (such as MySQL, PostgreSQL, or Oracle), or refine health monitoring algorithms without redeploying or risking the stability of the core execution orchestrator.

- **Availability:** High availability is guaranteed by deploying all core microservices, API gateways, and message broker nodes in redundant, load-balanced configurations across multiple cloud availability zones. This fault-tolerant structure guarantees ninety-nine point nine nine percent platform availability, ensuring that enterprise deployment pipelines are never blocked by orchestrator downtime.

# 6. Architectural Justification, Trade-offs, and Future Scope

## 6.1 Rationale and Trade-off Analysis

The choice of a hybrid microservices and event-driven architecture over traditional monolithic or simple

three-tier architectures is driven by the strict fault isolation and asynchronous processing demands of database DevOps. In a monolithic architecture, a memory leak or runtime crash in the SQL parsing engine or web reporting dashboard could crash the entire process, potentially interrupting an active schema migration midway through execution and corrupting production database tables. The microservices pattern completely isolates execution components from auxiliary platform services.

However, this architectural selection introduces well-defined engineering trade-offs. Distributed microservices and event-driven brokers increase overall operational complexity, requiring robust service discovery mechanisms, distributed tracing tools (such as OpenTelemetry and Jaeger), centralized logging stacks, and network overhead incurred by message broker serializations. Furthermore, managing distributed state machine consistency across multi-stage migrations requires careful implementation of compensation transactions. These trade-offs are fully justified because data integrity and zero application downtime take precedence over operational simplicity in enterprise database management.

## 6.2 Future Enhancements and Long-Term Maintainability

To maintain long-term relevance and adaptability, the platform architecture has been engineered with explicit extension points that accommodate future technological enhancements:

- **AI-Driven Predictive Risk Scoring:** Integrating machine learning models into the telemetry monitoring pipeline to analyze historical query execution plans, database traffic patterns, and table growth rates, enabling predictive detection of lock risks before executing DDL commands on multi-terabyte tables.

- **Cross-Cloud Multi-Region Schema Synchronization:** Expanding the orchestration engine's capabilities to manage complex, asynchronous schema evolution across distributed multi-region and multi-cloud database deployments, ensuring schema consistency across globally active-active database clusters.

- **Automated Data Backfill Generation:** Developing intelligent code-generation components capable of automatically drafting, testing, and verifying dual-write application middleware and data transformation scripts during complex column splitting or table normalization tasks.

## Rubrics for Calculation

| Evaluation Criteria | Excellent (4 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| **System Requirement Analysis (30%)** | Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture. | Analyzes most requirements with minor omissions or limited explanation. | Identifies only basic requirements with partial analysis. | Requirement analysis is incomplete, inaccurate, or lacks clarity. |
| **Selection of Software Architecture (25%)** | Selects the most appropriate architectural style with strong technical justification aligned to system requirements. | Selects a suitable architecture with reasonable justification. | Architecture is partially suitable with limited justification. | Inappropriate architecture selected or justification is missing. |
| **Architecture Diagram and Component Design (20%)** | Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented. | Diagram is mostly complete with minor omissions or notation issues. | Diagram includes only essential components and lacks sufficient detail. | Diagram is incomplete, unclear, or technically incorrect. |
| **Component Interaction and Quality Attribute Analysis (15%)** | Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability. | Explains most components and quality attributes with minor gaps. | Provides limited explanation of interactions and addresses only some quality attributes. | Component interactions and quality attributes are poorly explained or missing. |

| | | | | |
|---|---|---|---|---|
| **Architectural Justification and Technical Presentation (10%)** | Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report. | Provides adequate justification with minor gaps; report is well organized. | Limited justification with minimal discussion of trade-offs or future scope; report needs improvement. | Justification is weak or absent; report lacks organization and technical clarity. |

| Criteria | **Mark** |
|---|---|
| System Requirement Analysis **(30%)** | /5 |
| Selection of Software Architecture **(25%)** | /5 |
| Architecture Diagram and Component Design **(20%)** | /5 |
| Component Interaction and Quality Attribute Analysis **(15%)** | /5 |
| Architectural Justification and Technical Presentation **(10%)** | /5 |
| **TOTAL** | /25 |